

JOB INTELLIGENT

Plateforme de centralisation et recommandation d'offres d'emploi Data

Data Engineering & Power BI — Examen Final | Avril 2026

**Phase 1 : COMPLETE | Phase 2
: COMPLETE | Phases 3-6 : En
cours**

1. Contexte du Projet

Job Intelligent est un projet academique d'examen final visant a centraliser les offres d'emploi dans le domaine de la Data (Data Engineer, Data Scientist, Data Analyst, etc.) et a proposer un systeme de recommandation intelligent base sur le NLP.

L'architecture reproduit fidelement un pipeline Data industriel complet :

- Collecte multi-sources via APIs officielles et scraping HTML
- Stockage en Data Lake selon l'architecture Medallion (Bronze / Silver / Gold)
- Transformation distribuee avec Apache Spark
- Data Warehouse en schema etoile sous PostgreSQL
- Systeme de recommandation NLP (Sentence-BERT)
- Dashboards analytiques Power BI

2. Architecture Technique

2.1 Stack Technique

Composant	Outil	Role
Langage	Python 3.11	Developpement principal
Orchestration	Apache Airflow 2.9.0	Planification des pipelines ETL
Traitement	Apache Spark 3.5.1	Transformation distribuee Bronze -> Silver
Data Lake	MinIO	Stockage Bronze / Silver / Gold (compatible S3)
Data Warehouse	PostgreSQL 15	Stockage structure en schema etoile
Interface DB	pgAdmin 4	Administration PostgreSQL
Transformation	dbt	Modeles SQL versionnes (Phase 3)
NLP / ML	Sentence-BERT	Systeme de recommandation (Phase 4)
Visualisation	Power BI	Dashboards analytiques (Phase 5)
API	FastAPI	Endpoint de recommandation (Phase 4)
Conteneurs	Docker	Environnement reproductible via docker-compose

2.2 Architecture Data Lake — Medallion

Le Data Lake suit l'architecture Medallion a 3 couches stockees dans MinIO :

Couche	Bucket MinIO	Format	Description
Bronze	bronze/	JSON	Donnees brutes telles que recues des APIs
Silver	silver/	Parquet	Donnees nettoyees, normalisees et deduplicatees
Gold	gold/	Parquet/Tables	Donnees agregees pretes pour DWH et Power BI

2.3 Infrastructure Docker

Tous les services tournent dans des conteneurs Docker orchestrés via docker-compose :

Service	Port externe	Port interne	Statut
PostgreSQL	5433	5432	Operationnel
pgAdmin	5050	80	Operationnel
Apache Airflow	8082	8080	Operationnel
Spark Master UI	8083	8080	Operationnel
Spark Master	7077	7077	Operationnel
MinIO API	9002	9000	Operationnel
MinIO Console	9003	9001	Operationnel

3. Ce que Nous Avons Accompli

Phase 1 — Infrastructure & Collecte [COMPLETE]

3.1 Infrastructure Docker

- docker-compose.yml configure avec 7 services : PostgreSQL, pgAdmin, Airflow, Spark Master, 2 Workers, MinIO
- Gestion des conflits de ports (PostgreSQL sur 5433, Airflow sur 8082, MinIO sur 9002/9003)
- Volumes montes pour la persistance des donnees et le partage des scripts
- Cluster Spark : 1 master + 2 workers avec 2 cores / 2 Go RAM chacun

3.2 Sources de Donnees & Scrapers

Source	Methode	Profils	Offres	Statut
France-Travail	API OAuth2	6	421	Operationnel
Adzuna	API officielle	6	600	Operationnel
Emploi.ma	Scraping HTML	3	140	Operationnel
TOTAL	—	—	1 161	Complet

3.3 DAG Airflow

- Pipeline pipeline_collecte_offres orchestre la collecte quotidienne a 8h00
- Les 3 scrapers s'executent en parallele
- Gestion des retries automatiques (2 tentatives, delai 5 min)
- Donnees sauvegardees en JSON dans MinIO Bronze avec horodatage

3.4 Schema Data Warehouse

- Base dediee dwh_job_intelligent creee dans PostgreSQL
- Schema en etoile : fact_offres + 4 dimensions (dim_date, dim_entreprise, dim_lieu, dim_competence)
- Toutes les tables creees et validees

Phase 2 — Couche Silver Spark [COMPLETE]

3.5 Script PySpark spark_silver_transform.py

- Lecture des JSON depuis MinIO Bronze via connecteur S3A (hadoop-aws 3.3.4)
- Explosion des tableaux offres[] imbriques dans chaque fichier JSON
- Nettoyage : trim, suppression espaces multiples, filtres qualite
- Normalisation des types de contrat vers vocabulaire commun (CDI/CDD/Stage/Alternance/Freelance)
- Extraction salaire_min / salaire_max depuis les libelles texte
- Deduplication par offre_id avec fenetre temporelle (row_number)
- Ecriture en format Parquet partitionne par source dans MinIO Silver
- Table unifiee inter-sources avec deduplication et priorite (FranceTravail > Adzuna > Emploima)
- Rapport qualite automatique : taux de nulls sur colonnes critiques

3.6 Resultats Phase 2

Metrique	Valeur	Detail
Fichiers Bronze lus	6 fichiers	3 sources x plusieurs profils metiers
Offres extraites (explosion)	280 offres	Depuis tableaux offres[] imbriques
Offres apres nettoyage	29 offres	Apres filtrage et deduplication
Qualite donnees	100%	0% de nulls sur titre, entreprise, ville, type_contrat, date
Format sortie	Parquet	Partitionne par source dans silver/emploima/ et silver/unified/

4. Problemes Rencontres & Solutions

Probleme	Cause	Solution
ClassNotFoundException S3AFileSystem	JARs hadoop-aws absents	Copie manuelle dans /opt/spark/jars/ des 3 conteneurs
NoSuchMethodError IOUtils.release	Conflit versions SDK AWS	Remplacement par aws-java-sdk-bundle- 1.11.1026
NoSuchBucket bronze	Endpoint MinIO ignore	Passer les --conf directement dans spark- submit
UNRESOLVED_COLUMN id/titre	JSON enveloppe (tableau offres[])	Ajout F.explode('offres') avant select
Buckets MinIO vides	docker-compose down -v efface volumes	Recreer buckets + retrigger DAG Airflow
spark-submit not found	PATH non configure dans image Spark	Utiliser chemin complet /opt/spark/bin/spark-submit
Antislash PowerShell	PowerShell interprete \ litteralement	Commande sur une ligne ou backtick `

5. Roadmap & Prochaines Etapes

#	Phase	Description	Statut
1	Infrastructure & Collecte	Docker, scrapers, DAG Airflow, schema DWH, MinIO Bronze	COMPLETE
2	Couche Silver — Spark	Transformation PySpark, nettoyage, Parquet Silver, table unifiée	COMPLETE
3	Data Warehouse — dbt	Modeles dbt, chargement fact_offres + dimensions, tests qualite	PROCHAIN
4	NLP & FastAPI	Sentence-BERT, similarite cosinus, API /recommend	A faire
5	Power BI	Connexion PostgreSQL, dashboards marche + recommandations	A faire
6	Finalisation	Tests end-to-end, documentation finale, rapport et presentation	A faire

5.1 Phase 3 — Detail des prochaines actions

- Configurer dbt dans le projet (dbt init, profiles.yml pour PostgreSQL)
- Creer les modeles dbt : stg_emploima.sql, stg_adzuna.sql, stg_france_travail.sql
- Charger fact_offres depuis la table Silver unifiée
- Alimenter les dimensions : dim_date, dim_entreprise, dim_lieu, dim_competence
- Ecrire les tests de qualite dbt (not_null, unique, accepted_values)
- Renouveler les cle API France-Travail et Adzuna pour repeupler le Bronze
- Corriger transform_adzuna (meme fix explosion offres[] qu'Emploi.ma)

6. Justification des Choix Techniques

Pourquoi MinIO et non stockage local ?

MinIO est compatible S3 et tourne dans Docker. Il peut etre remplace par AWS S3 ou Azure Blob Storage en production sans changer une ligne de code — demontrant la scalabilite de l'architecture.

Pourquoi Spark Master + 2 Workers ?

Le mode cluster reproduit une vraie architecture industrielle de traitement distribue. Avec 25 Go de RAM disponibles, le cluster tourne sans contrainte et demontre la capacite a traiter des volumes de donnees importants.

Pourquoi Airflow plutot qu'APScheduler ?

Airflow est le standard industriel pour l'orchestration de pipelines Data. Il offre une interface web, la gestion des dependances entre taches, les retries automatiques, et un historique complet des executions.

Pourquoi 3 sources de donnees ?

La diversite des sources (API officielle France, API internationale, scraping Maroc) demontre la capacite a integrer des formats heterogenes — competence cle d'un Data Engineer. Le projet couvre ainsi deux pays et trois modalites d'accès aux donnees.

